

GUÍA DE ESTUDIO: JAVASCRIPT - ARRAYS

Métodos, funciones, propiedades y sintaxis que aparecen en los dos PDFs del profesor

Esta guía reúne el contenido de **UT_02.1_Arreglos** y **UT_02.2_Métodos de arrays**, sin añadir temas externos. Está pensada como hoja de repaso para el examen.

Idea clave: no memorices solo el nombre. Fíjate en si el método modifica el array original, qué devuelve y si recibe una función.

1. Métodos, propiedades y funciones principales

Nombre	¿Para qué sirve?	Ejemplo de utilización
Array() - constructor	Crea un array. Puede recibir valores o una longitud.	<pre>const nombres = new Array("Juan", "Ana");</pre>
[] - notación literal	Forma habitual de crear un array.	<pre>const numeros = [1, 2, 3];</pre>
length - propiedad	Devuelve cuántos elementos tiene el array.	<pre>const nums = [10, 20, 30]; console.log(nums.length); // 3</pre>
array[indice] - acceso	Permite acceder a un elemento mediante su posición. Los índices empiezan en 0.	<pre>const frutas = ["manzana", "pera"]; console.log(frutas[0]); // manzana</pre>
array[indice] = valor	Permite modificar un elemento existente.	<pre>let nums = [1, 2, 3]; nums[1] = 20;</pre>
push()	Añade uno o varios elementos al final y devuelve la nueva longitud.	<pre>let a = [1, 2]; a.push(3); // [1, 2, 3]</pre>
pop()	Elimina y devuelve el último elemento.	<pre>let a = [1, 2, 3]; let x = a.pop(); // x = 3</pre>
shift()	Elimina y devuelve el primer elemento.	<pre>let a = [1, 2, 3]; let x = a.shift(); // x = 1</pre>
unshift()	Añade uno o varios elementos al principio y devuelve la nueva longitud.	<pre>let a = [2, 3]; a.unshift(1); // [1, 2, 3]</pre>
splice()	Sirve para insertar, eliminar o sustituir elementos en cualquier posición.	<pre>let a = [1, 2, 3]; a.splice(1, 1, 10); // [1, 10, 3]</pre>
slice()	Copia una parte del array y devuelve un nuevo array. No modifica el original.	<pre>let a = [1, 2, 3, 4]; let b = a.slice(1, 3); // [2, 3]</pre>
concat()	Une arrays o valores y devuelve un nuevo array.	<pre>let a = [1, 2]; let b = a.concat([3, 4]); // [1, 2, 3, 4]</pre>
forEach()	Ejecuta una función una vez por cada elemento.	<pre>[1, 2, 3].forEach(n => console.log(n));</pre>
indexOf()	Busca la primera posición de un elemento. Si no existe devuelve -1.	<pre>["a", "b", "c"].indexOf("b"); // 1</pre>
lastIndexOf()	Busca la última posición en la que aparece un elemento.	<pre>["a", "b", "a"].lastIndexOf("a"); // 2</pre>
includes()	Comprueba si un elemento existe. Devuelve true o false.	<pre>[1, 2, 3].includes(2); // true</pre>
find()	Busca el primer elemento que cumpla una condición. Si no encuentra ninguno devuelve undefined.	<pre>let a = [2, 4, 7, 8]; a.find(n => n % 2 !== 0); // 7</pre>
findIndex()	Igual que find(), pero devuelve la posición del primer elemento que cumple la condición.	<pre>[2, 4, 7].findIndex(n => n > 5); // 2</pre>
findLastIndex()	Busca desde el final y devuelve el índice del último elemento que cumple la condición.	<pre>[2, 7, 4, 7].findLastIndex(n => n === 7); // 3</pre>

Nombre	¿Para qué sirve?	Ejemplo de utilización
filter()	Devuelve un nuevo array con todos los elementos que cumplen una condición.	<pre>[1, 2, 3, 4].filter(n => n > 2); // [3, 4]</pre>
map()	Crea un nuevo array transformando cada elemento.	<pre>[1, 2, 3].map(n => n * 2); // [2, 4, 6]</pre>
sort()	Ordena los elementos del array. Puede recibir una función para definir cómo ordenar.	<pre>let a = [10, 2, 5]; a.sort((x, y) => x - y); // [2, 5, 10]</pre>
reverse()	Invierte el orden de los elementos.	<pre>let a = [1, 2, 3]; a.reverse(); // [3, 2, 1]</pre>
split() - strings	Divide un texto y lo convierte en un array.	<pre>"Juan,Ana,Pedro".split(","); // ["Juan", "Ana", "Pedro"]</pre>
join()	Hace lo contrario de split: une los elementos del array en un string.	<pre>["Juan", "Ana"].join(", "); // "Juan, Ana"</pre>
reduce()	Recorre el array y combina sus elementos para obtener un único resultado.	<pre>[1, 2, 3, 4].reduce((total, n) => total + n, 0); // 10</pre>
reduceRight()	Hace lo mismo que reduce(), pero recorriendo el array de derecha a izquierda.	<pre>[1, 2, 3].reduceRight((total, n) => total + n, 0); // 6</pre>
Array.isArray()	Comprueba si un valor es realmente un array. Devuelve true o false.	<pre>Array.isArray([1, 2]); // true</pre>
some()	Comprueba si al menos un elemento cumple una condición.	<pre>[1, 3, 4].some(n => n % 2 === 0); // true</pre>
every()	Comprueba si todos los elementos cumplen una condición.	<pre>[2, 4, 6].every(n => n % 2 === 0); // true</pre>
fill()	Rellena parte del array repitiendo un mismo valor.	<pre>[1, 2, 3, 4].fill(0, 1, 3); // [1, 0, 0, 4]</pre>
copyWithin()	Copia elementos dentro del propio array, sobrescribiendo los que ya estaban.	<pre>[1, 2, 3, 4].copyWithin(0, 2, 4); // [3, 4, 3, 4]</pre>
flat()	Convierte un array multidimensional en uno más plano.	<pre>[1, [2, 3], [4]].flat(); // [1, 2, 3, 4]</pre>
flatMap()	Combina una transformación tipo map() con un aplanamiento del resultado.	<pre>[1, 2, 3].flatMap(n => [n, n * 2]); // [1, 2, 2, 4, 3, 6]</pre>
toString()	Convierte el array en una cadena de texto.	<pre>[1, 2, 3].toString(); // "1,2,3"</pre>
toLocaleString()	Convierte los elementos a texto teniendo en cuenta la configuración regional.	<pre>[1234567.89].toLocaleString("es-ES"); // "1.234.567,89"</pre>
valueOf()	Devuelve el valor del propio array.	<pre>const a = [1, 2, 3]; console.log(a.valueOf()); // [1, 2, 3]</pre>
localeCompare() - strings	Compara dos cadenas, especialmente útil para ordenar texto según la configuración regional.	<pre>["Ana", "Álvaro"].sort((a, b) => a.localeCompare(b));</pre>

2. Sintaxis y conceptos que también aparecen en los PDFs

Nombre	¿Para qué sirve?	Ejemplo de utilización
for	Bucle clásico para recorrer un array mediante índices.	<pre>for (let i = 0; i < a.length; i++) console.log(a[i]);</pre>
for...of	Recorre directamente los valores de un array.	<pre>for (const fruta of frutas) console.log(fruta);</pre>
typeof	Permite saber el tipo de un valor, aunque un array devuelve "object".	<pre>typeof [1, 2, 3]; // "object"</pre>
delete	Puede eliminar una posición de un array, pero deja un hueco y no reduce length.	<pre>let a = [1, 2, 3]; delete a[1];</pre>
Función de comparación	Se utiliza con sort() para decidir qué elemento va antes o después.	<pre>a.sort((x, y) => x - y);</pre>
thisArg	Parámetro opcional de métodos como find, filter y map para indicar el valor de this dentro de la función.	<pre>usuarios.filter(array.canJoin, array);</pre>
Symbol.isConcatSpreadable	Hace que concat() trate un objeto parecido a un array como si fuera un array y extienda sus elementos.	<pre>obj[Symbol.isConcatSpreadable] = true;</pre>

3. Diferencias que debes tener claras

Tema	Opción A	Opción B
Añadir / quitar	push() -> añade al final	pop() -> quita del final
Añadir / quitar	unshift() -> añade al principio	shift() -> quita del principio
Copiar / modificar	slice() -> crea otro array y no modifica el original	splice() -> modifica el original
Buscar	find() -> devuelve un elemento	filter() -> devuelve un array con todos los que cumplen
Transformar / recorrer	map() -> crea un nuevo array transformado	forEach() -> ejecuta una función y no crea ese nuevo array
Comprobar contenido	indexOf() -> devuelve posición	includes() -> devuelve true/false
Resultado final	reduce() -> combina elementos en un único resultado	reduceRight() -> igual, pero de derecha a izquierda

4. Los métodos que más conviene dominar

push() · pop() · shift() · unshift() · splice() · slice() · concat() · forEach() · indexOf() · lastIndexOf() · includes() · find() · findIndex() · filter() · map() · sort() · reverse() · split() · join() · reduce() · Array.isArray()

Como ampliación, el PDF también incluye findLastIndex(), reduceRight(), some(), every(), fill(), copyWithin(), flat() y flatMap().

5. Tres reglas que suelen causar errores

1. push() añade al final, pop() elimina del final, unshift() añade al principio y shift() elimina del principio.
2. splice(), sort() y reverse() modifican el array original.
3. slice() no modifica el original y devuelve un nuevo array.

Fuentes: UT_02.1_Arreglos.pdf y UT_02.2_Array_Methods.pdf, proporcionados en la conversación.